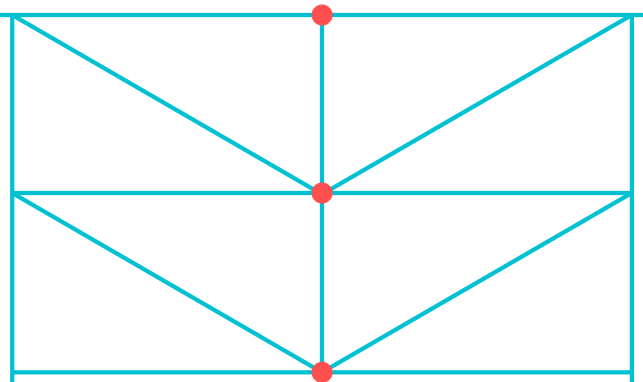
Parallelization of Solvers

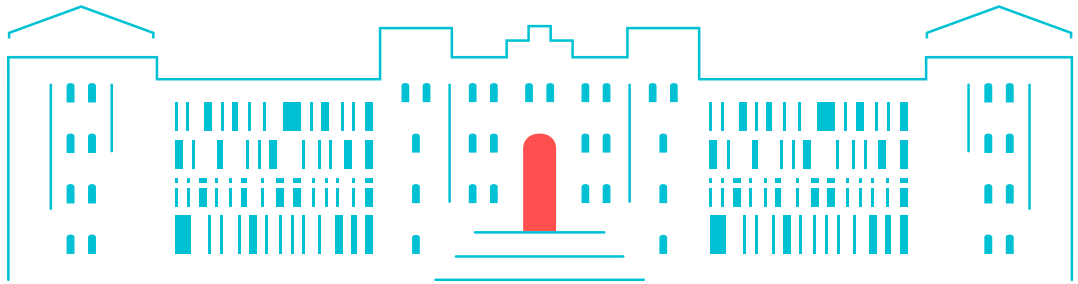
Dr.-Ing. Fynn Bensel

Chair Computational Mathematics
Institute of Mathematics (E-10)
Building E, Room 3.045

TUHH
Hamburg
University of
Technology



16.04.2026



01 - Introduction

Literature

TUHH

Main literature

- Numerical linear algebra for high-performance computers. Society for Industrial and Applied Mathematics, Dongarra, J. J. 1998: https://katalog.tub.tuhh.de/Record/1651240000
- Parallel Programming : for Multicore and Cluster Systems. Rauber, T., Rünger, G. 2010: https://katalog.tub.tuhh.de/Record/1649977417

Further reading

- Programming multicore and many-core computing systems. Xhafa, F., Pllana, S. 2017: https://katalog.tub.tuhh.de/Record/1679551701
- Shared memory application programming : concepts and strategies in multicore application programming. Alessandrini, V. 2016: https://katalog.tub.tuhh.de/Record/879391162
- Using OpenMP - the next step : affinity, accelerators, tasking, and SIMD. Pas, R. v. d., Terboven, C., Stotzer, E. 2017: https://katalog.tub.tuhh.de/Record/1066892318

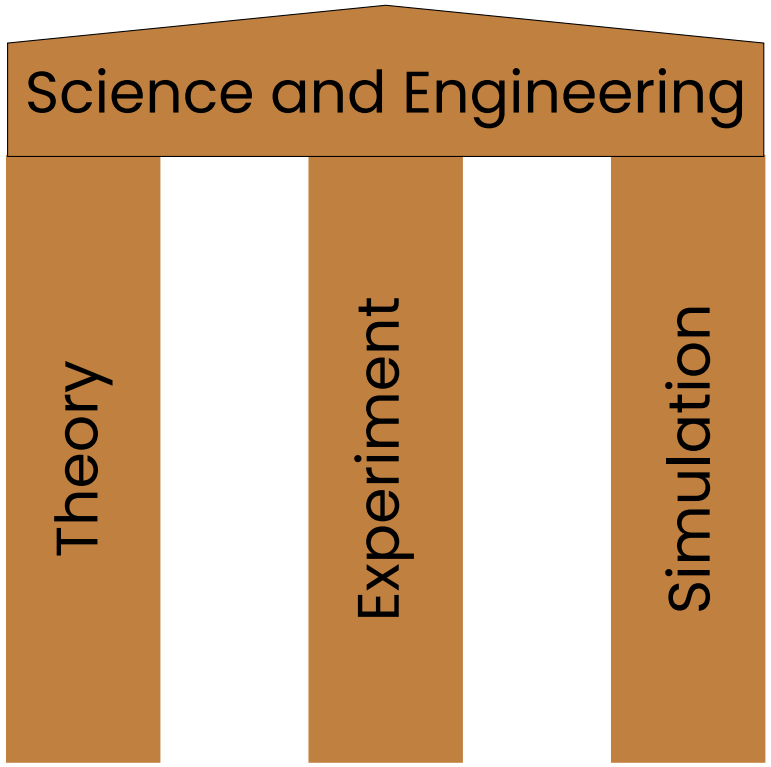
Learning objectives

TUHH

- Why do we build parallel computers? Moore’s law and a "fundamental turn towards concurrency".
- How is a parallel computer build?
- Classification of parallel computers: Flynn’s taxonomy
- Programming paradigms: Shared versus distributed memory and multi-threading versus message passing
- Vocabulary: processes, threads, speed-up, efficiency, ...

Why computer simulations?

TUHH

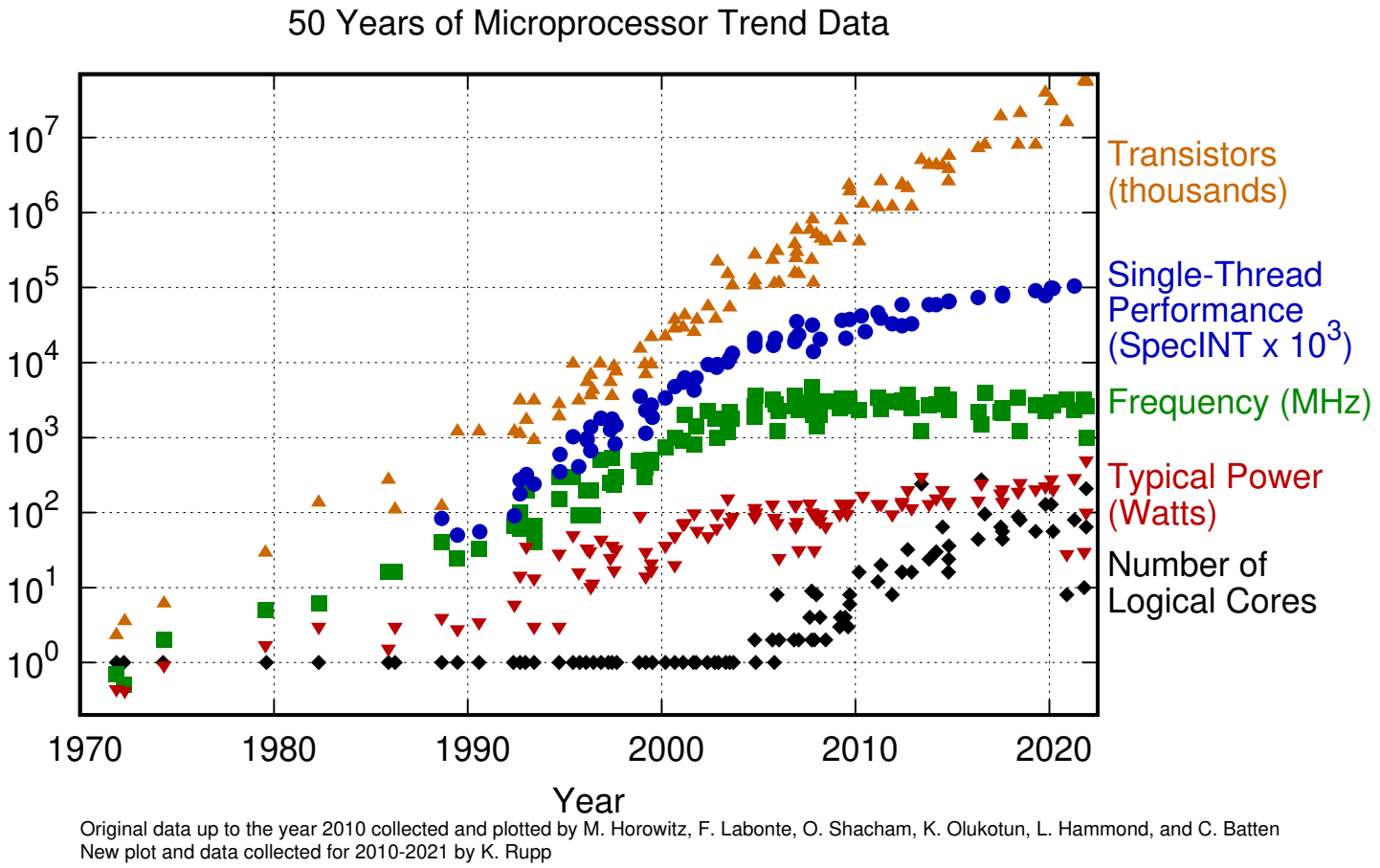


- High performance computing systems required to run such simulations

- Computational modeling: third pillar of science
- Many models are extremely complex: weather forecast, crash simulations, drug design, computer graphics applications

Forschungszentrum Jülich, https://upload.wikimedia.org/wikipedia/commons/0/06/J%C3%BClich_Wizard_for_European_Leadership_Science_%28JUWELS%29.jpg
CC BY-SA 4.0, via Wikimedia Commons

Why parallel programming?



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Moore's law continues to hold

Since around 2005 because of more and more cores per CPU instead of faster clock speed
$\rightarrow$ utilizing many-core CPUs requires **parallelism**.

Source: https://github.com/karlrupp/microprocessor-trend-data/blob/master/50yrs/50-years-processor-trend.pdf

State of the art HPC systems

TUHH

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	**Eagle** - Microsoft [illegible]v5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H[illegible] Azure Microsoft Azure United States	2,073,600	561.20	[illegible]	
6	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461

That is a lot of **energy**

That is a lot of **cores**

GPUs have become an important component of HPC systems

https://www.top500.org/lists/top500/list/2025/11/

Flynn's Taxonomy

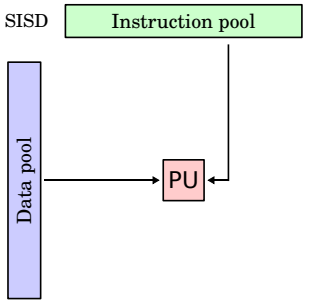
TUHH

Räuber and Bünger, Section 2.2, page 10.

"[...] a parallel computer can be characterized as a collection of processing elements that can communicate and cooperate to solve large problems fast"

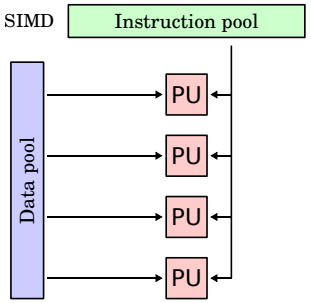
Single **I**nstruction **S**ingle **D**ata

Old personal computer



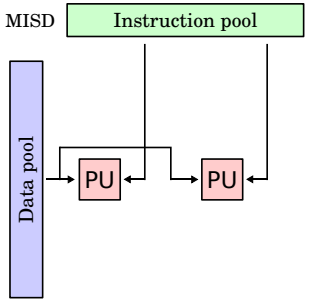
Single **I**nstruction **M**ultiple **D**ata

computer graphics algorithm



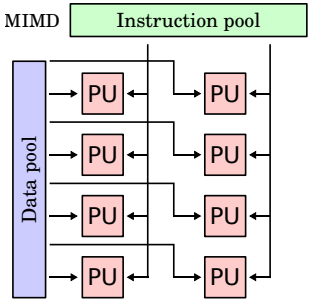
Multiple **I**nstruction **S**ingle **D**ata

Space Shuttle flight control computer



Multiple **I**nstruction **M**ultiple **D**ata

Multicore processors or cluster system



https://en.wikipedia.org/wiki/Flynn%27s_taxonomy

Tasks and Granularity

TUHH

Task

A part of a larger computation that can be run in parallel with other tasks.

Example: Compute $1 + 2 + 3 + 4$

Tasks and Granularity

Task

A part of a larger computation that can be run in parallel with other tasks.

Example: Compute $1 + 2 + 3 + 4$

- Task 1: $1 + 2$
- Task 2: $3 + 4$
- Task 3: Result from Task 1 + Result from Task 2

Tasks and Granularity

TUHH

Task

A part of a larger computation that can be run in parallel with other tasks.

Example: Compute $1 + 2 + 3 + 4$

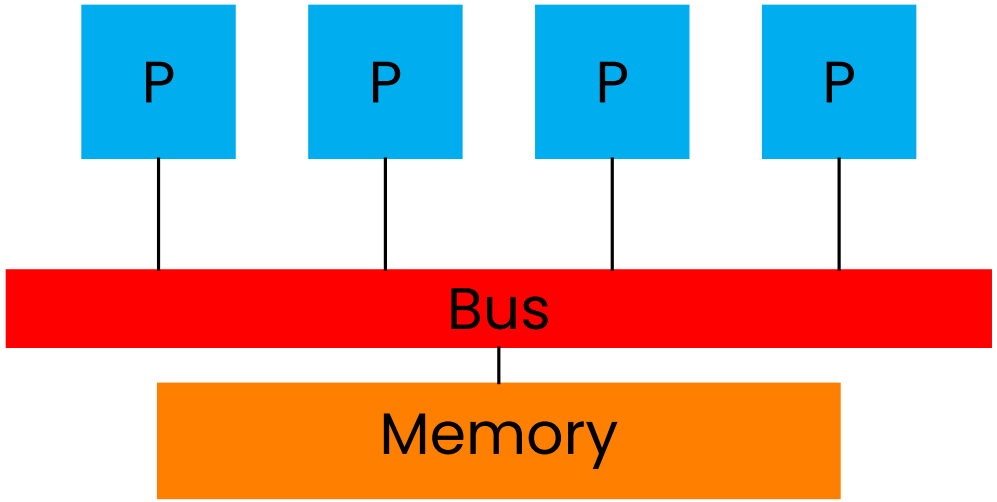
- Task 1: $1 + 2$
- Task 2: $3 + 4$
- Task 3: Result from Task 1 + Result from Task 2

Granularity

Refers to the size of tasks with respect to the number of CPU instructions.

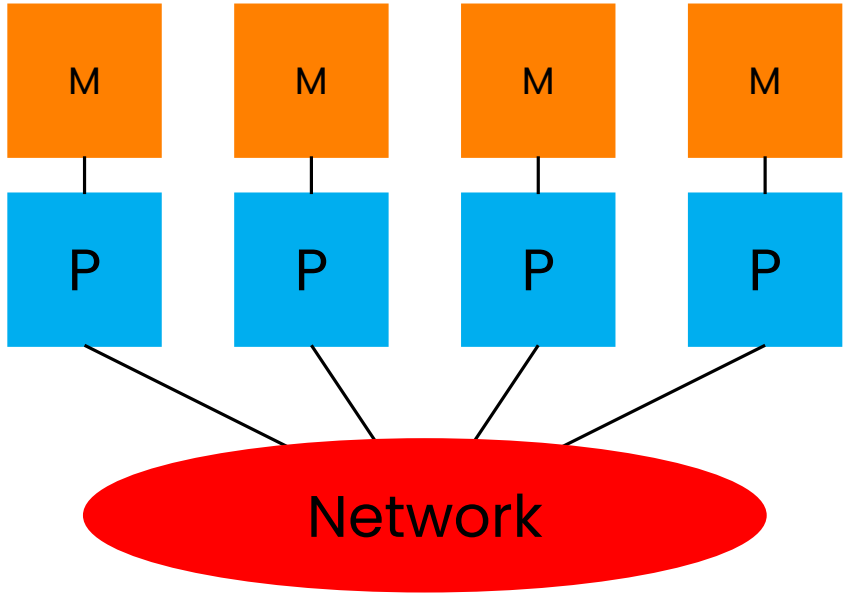
A decomposition into many small tasks is called fine grained.
$(1 + 2) + (3 + 4) + (5 + 6) + (7 + 8) + (9 + 10)$
A decomposition into fewer larger tasks is called coarse grained.
$(1 + 2 + 3 + 4 + 5) + (6 + 7 + 8 + 9 + 10)$

Shared vs distributed memory



Shared memory

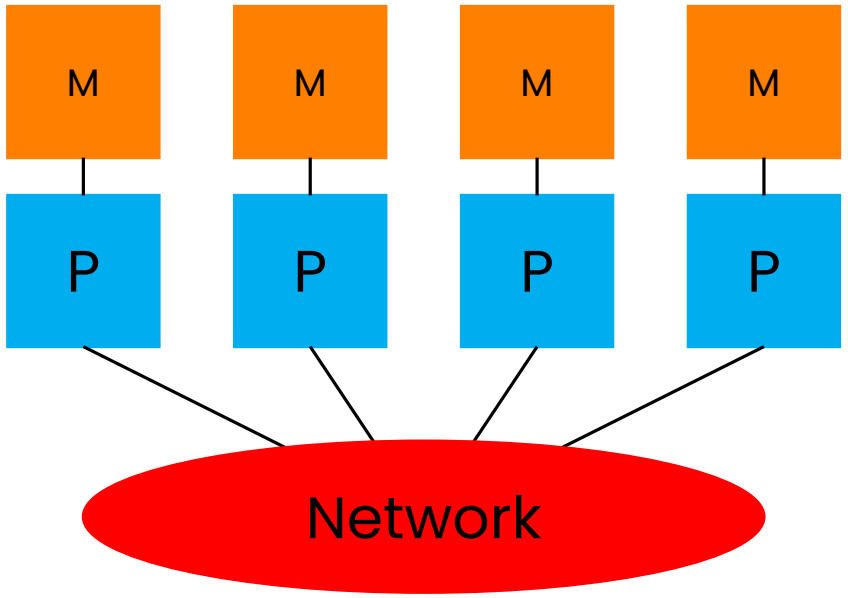
- Single address space
- All processes have access to the pool of shared memory



Distributed memory

- Each processor has its own local memory
- Message-passing is used to exchange data between processors

Computers with distributed memory



- **Modern HPC systems:** many compute nodes, linked by a network
- Every node has multiple CPUs with multiple cores, and potentially one or more GPUs or other accelerators
- CPUs/cores of a node share physical memory (except maybe low level cache) Data transfer between nodes requires message passing through network
- It is, however, possible to run multiple MPI processes on a single node

Processes vs threads

TUHH

Process

A **process** executes a series of instructions on its own. It typically has its own logical memory that cannot be seen by other processes. It can be assigned to a core, CPU or node and can run multiple threads.

Can be managed via the **Message Passing Interface (MPI)**

Thread

A **thread** also executes a series of instructions but typically shares a logical memory with other thread.

Can be managed via the **OpenMP** standard.

Message passing vs OpenMP

TUHH

Distributed memory / MPI

- Need to code communication explicitly
- Intrusive, need to modify code
- Message passing can increase memory footprint

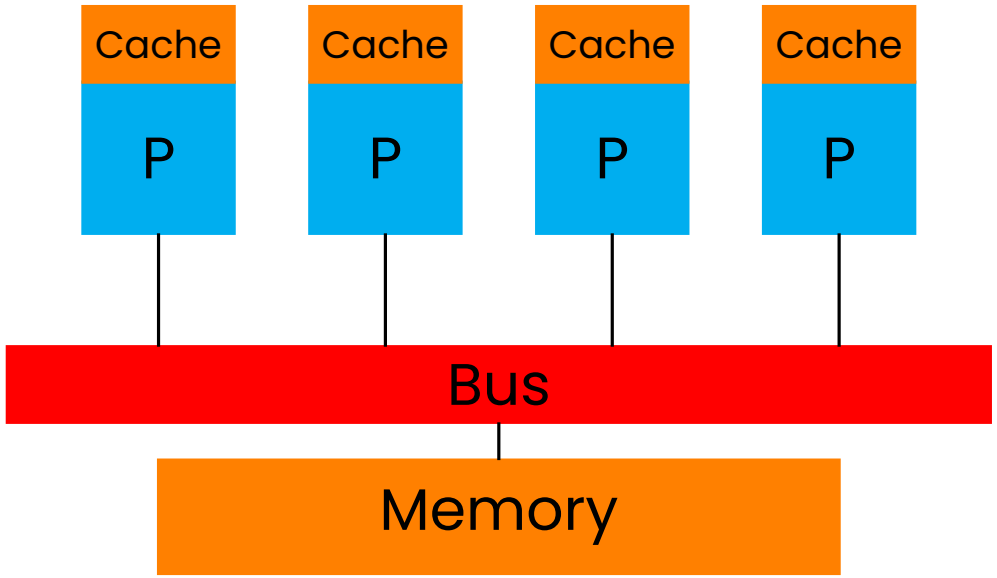
Shared memory / OpenMP

- No extra communication needed
- Theoretically just needs annotations of code...
- ...but lots of hidden traps to get good performance

...many codes today combine both to hybrid parallelism:
every MPI process spawns multiple OpenMP threads.

Cache coherence

TUHH



- CPU adjacent Cache holds a subset of shared memory
- If one processor changes its cache, the others need updating to maintain coherence

- This happens automatically but can impact shared memory parallel performance (False sharing, Cache thrashing)

Race conditions

TUHH

Race condition

A shared memory programme where the result depends on the order in which threads perform operations.

… the order in which threads do things is essentially random, making the result random.

This is a **bug** and should never happen. Race conditions can be hard to detect and debug though.

Race conditions

TUHH

Example

Assume we want to compute $1 + 2 + 3 + \cdots + n$ for some large n but we forgot Gauss's handy formula.

We code:

```c
// This we only need to write output
#include <stdio.h>

int main(int argc, char **argv)
{
    int sum = 0;

    int n = 5e+6;

    for (int i = 0; i < n; i++)
    {
        sum += (i + 1);
    }

    printf("\n");
    printf("Gauss says, the answer should be %i \n", (int)(n / 2) * (n + 1)
   );
    printf("Our answer is:                   %i \n", sum);
}
```

Race conditions

TUHH

Example

Assume we want to compute $1 + 2 + 3 + \cdots + n$ for some large n but we forgot Gauss's handy formula.

We code:

```
1  // Include the OpenMP header
2  #include <omp.h>
3
4  // This we only need to write output
5  #include <stdio.h>
6
7  int main(int argc, char **argv)
8  {
9      int sum = 0;
10
11     int n = 5e+6;
12
13     #pragma omp parallel for
14     for (int i = 0; i < n; i++)
15     {
16         sum += (i + 1);
17     }
18
19     printf("\n");
20     printf("Gauss says, the answer should be %i \n", (int)(n / 2) * (n + 1)
       );
21     printf("Our answer is:                  %i \n", sum);
22 }
```

and parallelize
(more next week)

Result

Every run gives a different result!
The answer seems to be random...

Speedup and parallel efficiency

TUHH

Speedup

$$S(P) = \frac{\text{Runtime in serial}}{\text{Runtime in parallel using P cores}}$$

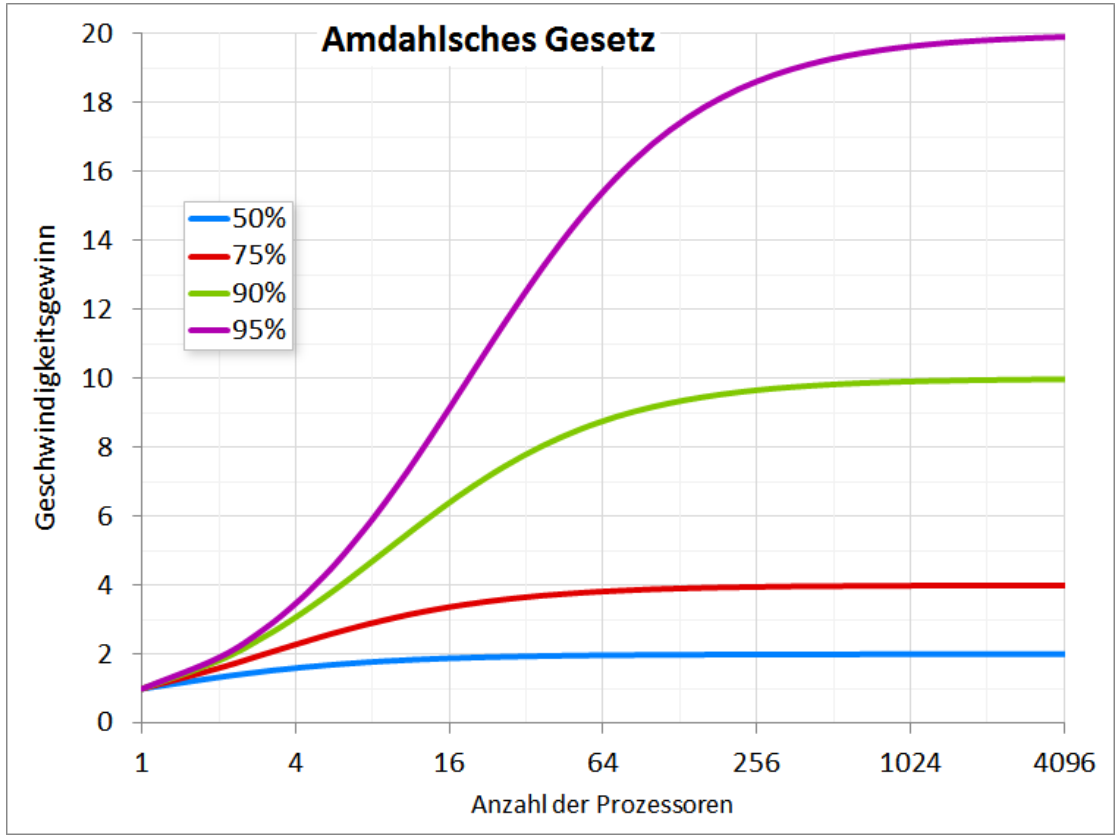
Efficiency

$$E(P) = \frac{S(P)}{P}$$

Example

A program takes 14.3 seconds to run on a single core and 4.7 seconds to run on 5 cores.
We have $S(5) = 14.3/4.7 \approx 3.04$ with parallel efficiency $E(5) = 3.04/5 \approx 0.608 = 60.8\%$
Note that $E(P)$ can normally not be better than 100%.

Ahmdahl's law

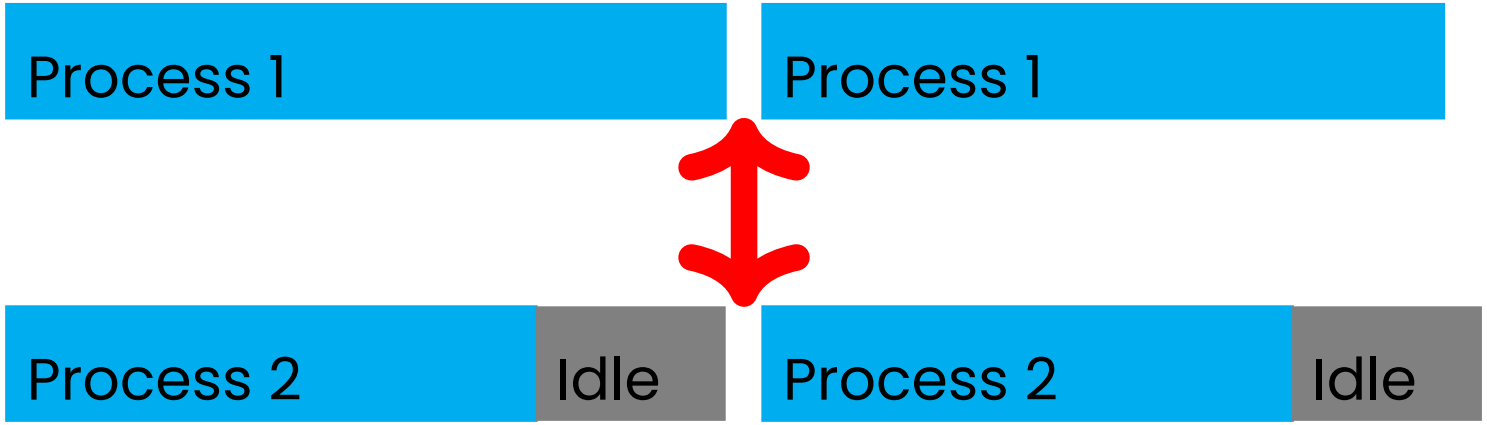


Let f be the percentage of a program that run in serial. Then: $S(P) \leq \frac{1}{f}$
$f = 10\%$ means speedup can never be better than $\frac{1}{0.1} = 10$

Daniels220 at English Wikipedia, CC BY-SA 3.0 via Wikimedia Commons. https://commons.wikimedia.org/wiki/File:AmdahlsLaw.svg

Load balancing

TUHH



Largest speedup is typically achieved if all tasks are of equal “size”.
Tasks of different size give rise to load imbalances which can reduce performance.

Hello World

TUHH

```c
// This we only need to write output
#include <stdio.h>

int main(int argc, char** argv)
{
    #pragma omp parallel
    {
        printf("Hello World\n");
    }
}
```

Compile:

```
    gcc hello_world.c -o hello_world.out

```

Run

```
    ./hello_world.out

```

Hello World OpenMP

TUHH

```c
// Include the OpenMP header
#include <omp.h>

// This we only need to write output
#include <stdio.h>

int main(int argc, char** argv)
{
    // Begin the parallel part of the code: here, the OpenMP library will spawn a number of threads equal
    // to the OMP_NUM_THREADS environment variable. Each thread will run the code inside the brackets.
    #pragma omp parallel
    {

        // This integer will store the number (rank) of the current thread
        int thread_num;

        // This integer will store the total number of threads
        int num_threads;

        // get the number of the current thread
        thread_num = omp_get_thread_num();

        // get the total number of threads
        num_threads = omp_get_num_threads();

        printf("Hello World... from thread number = %d out of %d many threads \n", thread_num, num_threads);
    }
    // Ending of parallel region

    // Important: if you ask how many threads there are outside of the parallel region, the answer will be one!
    printf("Number of threads outside of parallel region: %d \n", omp_get_num_threads());
}
```

Hello World MPI

TUHH

```c
// This needs to be included to make the MPI library available in your program
#include <mpi.h>

// This we only need to write output
#include <stdio.h>

int main(int argc, char** argv) {

    // Integer to store the number of MPI processes launched
    int world_size;

    // The rank ("number") of the current process: from 0 to world_size-1
    int world_rank;

    // Initialize the MPI environment: we need no input parameters, hence NULL
    MPI_Init(NULL, NULL);

    // MPI_Comm_Size is an MPI command that determines the number of processes. This number
    // is set by the -n flag in mpirun. This number is the same for all processes.
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Get the rank of the process: note that every process will be running a copy of this program
    // so in every process, world_rank will have a different value
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Print "Hello World". Note that every process will print this, so you will see world_size many messages in your terminal
    printf("Hello world says rank %d out of %d processes \n", world_rank, world_size);

    // Finalize the MPI environment.
    MPI_Finalize();
}
```

Summary

You should now be familiar with the following terms:

- Task, granularity, Flynn's taxonomy
- shared/distributed memory, tasks/processes
- Cache coherence and race conditions
- Speedup and parallel efficiency
- Amdahl's law
- Load balancing
- Initialization of MPI / OpenMP in a C program

Homework